# System Analysis & Design

## An Object-Oriented Approach with UML

*Fifth Edition*

**Alan Dennis**
*Indiana University*

**Barbara Haley Wixom**
*Massachusetts Institute of Technology*

**David Tegarden**
*Virginia Tech*
*With contributions by Elaine Seeman,*
*East Carolina University*

WILEY

## Navigation Structure Design

The navigation structure defines the basic components of the interface and how they work together to provide functionality to users. A *windows navigation diagram (WND)*[6] is used to show how all the screens, forms, and reports used by the system are related and how the user moves from one to another. Most systems have several WNDs, one for each major part of the system.

A WND is very similar to a behavioral state machine (see Chapter 6), in that they both model state changes. A behavioral state machine typically models the *state* changes of an object, whereas a WND models the state changes of the user interface. In a WND, each state of the user interface is represented as a box. A box typically corresponds to a user interface component, such as a *window, form*, *button,* or *report*. For example, in Figure 10-4, there are five separate states: Client Menu, Find Client Form, Add Client Form, Client List, and Client Information Report.

*Transitions* are modeled as either a single-headed or double-headed arrow. A single-headed arrow indicates that a return to the calling state is not required, whereas a double-headed arrow represents a required return. For example in Figure 10-4, the transition from the Client Menu state to the Find Client Form state does not require a return. The arrows are labeled with the action that causes the user interface to move from one state to another. For example, in Figure 10-4, to move from the Client Menu state to the Find Client Form state, the user must click the Find Client Button on the Client Menu.

The last item to be described in a WND is the *stereotype.* A stereotype is modeled as a text item enclosed within guillemets or angle brackets (<< >>). The stereotype represents the type of user interface component of a box on the diagram. For example, the Client Menu is a window, whereas Find Client Form is a form.

The basic navigation structure of an interface follows the basic structure of the business process itself, as defined in the use cases and behavioral model. The analyst starts with the essential use cases and develops the fundamental flow of control of the system as it moves from object to object. The analyst then examines the use scenarios to see how well the WND

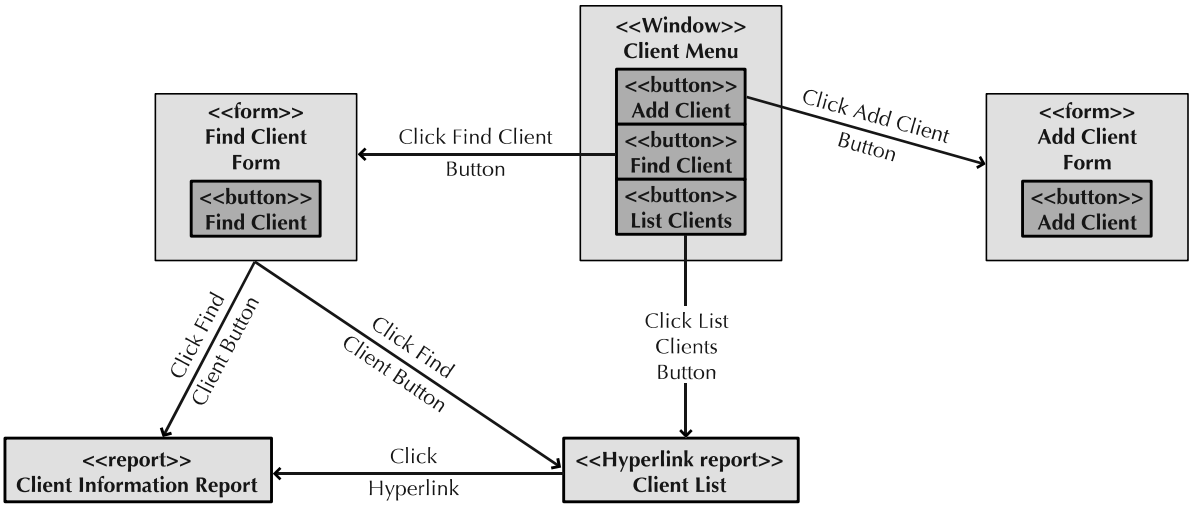



**FIGURE 10-4** Sample WND

[6] A WND is based on the behavioral state machine and object diagrams [see Meilir Page-Jones, *Fundamentals of Object-Oriented Design in UML* (New York: Dorset House, 2000)].

supports them. Quite often, the use scenarios identify paths through the WND that are more complicated than they should be. The analyst then reworks the WND to simplify the ability of the interface to support the use scenarios, sometimes by making major changes to the menu structure, sometimes by adding shortcuts.

## Interface Standards Design

*Interface standards* are the basic design elements that are common across the individual screens, forms, and reports within the system. Depending on the application, there may be several sets of interface standards for different parts of the system (e.g., one for Web screens, one for mobile screens, one for paper reports, one for input forms). For example, the part of the system used by data entry operators might mirror other data entry applications in the company, whereas a Web interface for displaying information from the same system might adhere to some standardized Web format. Likewise, each individual interface might not contain all of the elements in the standards (e.g., a report screen might not have an edit capability), and they might contain additional characteristics beyond the standard ones, but the standards serve as the touchstone that ensures the interfaces are consistent across the system. The following sections discuss some of the main areas in which interface standards should be considered: metaphors, objects, actions, icons, and templates.

**Interface Metaphor** First of all, the analysts must develop the fundamental interface metaphor(s) that defines how the interface will work. An interface metaphor is a concept from the real world that is used as a model for the computer system. The metaphor helps the user understand the system and enables the user to predict what features the interface might provide, even without actually using the system. Sometimes systems have one metaphor, whereas in other cases there are several metaphors in different parts of the system.

Often, the metaphor is explicit. Quicken, for example, uses a checkbook metaphor for its interface, even to the point of having the users type information into an on-screen form that looks like a real check. In other cases, the metaphor is implicit or unstated, but it is there, nonetheless. Many Windows systems use the paper form or table as a metaphor.

In some cases, the metaphor is so obvious that it requires no thought. For example, most online stores use a shopping cart metaphor to temporarily store the items that the customer is considering purchasing. In other cases, a metaphor is hard to identify. In general, it is better not to force a metaphor that really doesn't fit a system, because an ill-fitting metaphor will confuse users by promoting incorrect assumptions.

**Interface Templates** An *interface template* defines the general appearance of all screens in the information system and the paper-based forms and reports that are used. The template design, for example, specifies the basic layout of the screens (e.g., where the navigation area(s), status area, and form/report area(s) will be placed) and the color scheme(s) that will be applied. It defines whether windows will replace one another on the screen or will cascade over the top of each other. The template defines a standard placement and order for common interface actions (e.g., File Edit View rather than File View Edit). In short, the template draws together the other major interface design elements: metaphors, objects, actions, and icons.

**Interface Objects** The template specifies the names that the interface will use for the major *interface objects,* the fundamental building blocks of the system, such as the classes. In many cases, the object names are straightforward, such as calling the shopping cart the "shopping cart." In other cases, it is not so simple. For example, Amazon.com sells much more than books. In some cases, the user might not know whether he or she is looking for a book,

CD, DVD, or Kindle download. In those cases, the user can use a catchall search item: All Departments. In the case that the user knows the type of item that he or she wants to buy, the user can limit the search by specifying more-specific types of search items, such as Apps for Android, Books, Kindle Store, or Music. Obviously, the object names should be easily understood and help promote the interface metaphor.

In general, in cases of disagreements between the users and the analysts over names, whether for objects or actions (discussed later), the users should win. A more understandable name always beats a more precise or more accurate one.

**Interface Actions** The template also specifies the navigation and command language style (e.g., menus) and grammar (e.g., object-action order; see the navigation design section later in this chapter). It gives names to the most commonly used *interface actions* in the navigation design (e.g., buy versus purchase or modify versus change).

**Interface Icons** The interface objects and actions and their status (e.g., deleted or overdrawn) may be represented by *interface icons.* Icons are pictures that appear on command buttons as well as in reports and forms to highlight important information. Icon design is very challenging because it means developing a simple picture less than half the size of a postage stamp that needs to convey an often-complex meaning. The simplest and best approach is to simply adopt icons developed by others (e.g., a blank page to indicate create a new file, a diskette to indicate save). This has the advantage of quick icon development, and the icons might already be well understood by users because they have seen them in other software.

Commands are actions that are especially difficult to represent with icons because they are in motion, not static. Many icons have become well known from widespread use, but icons are not as well understood as first believed. Use of icons can sometimes cause more confusion than insight. Icon meanings become clearer with use, but sometimes a picture is not worth even one word; when in doubt, use a word, not a picture.

## Interface Design Prototyping

An interface design prototype is a mock-up or a simulation of a computer screen, form, or report. A prototype is prepared for each interface in the system to show the users and the programmers how the system will perform. In the "old days," an interface design prototype was usually specified on a paper form that showed what would be displayed on each part of the screen. Paper forms are still used today, but more and more interface design prototypes are being built using computer tools instead of paper. The four most common approaches to interface design prototyping are storyboards, windows layout diagrams, and language prototypes.

**Windows Layout Diagram** A *windows layout diagram* is simply a picture that resembles the actual user interface that the user will gradually receive. Typically, it is created using a tool such as Microsoft's Visio. Using this type of tool, the designer can quickly drag and drop the user interface components onto the canvas to lay out the design of the user interface. For example, Figure 10-5 portrays a simple windows layout diagram. Even though there is no executable capability associated with a windows layout diagram, it does allow the user to quickly get a feel for the look of the user interface that will be delivered.

**Storyboard** At its simplest, an interface design prototype is a paper-based *storyboard.* The storyboard shows hand-drawn pictures of what the screens will look like and how they flow from one screen to another, in the same way a storyboard for a cartoon shows how the action will flow from one scene to the next (see Figure 10-6). Storyboards are the simplest technique

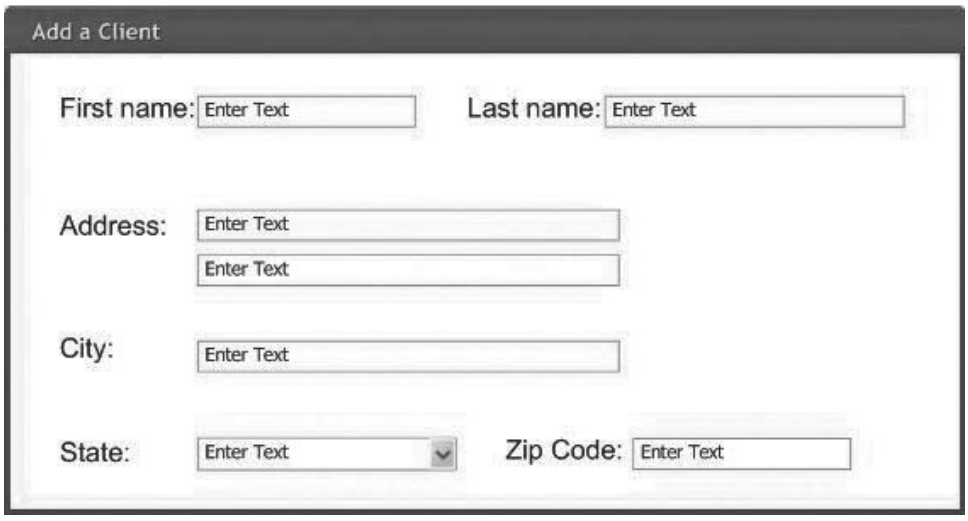



**FIGURE 10-5**
Sample Windows Layout Diagram

because all they require is paper (often a flip chart) and a pen—and someone with some artistic ability. Storyboards also combine both the navigation information of the windows navigation diagram and to some degree the layout information of the windows layout diagram. However, with today's graphics tools, the designer can work effectively with a set of users to design both

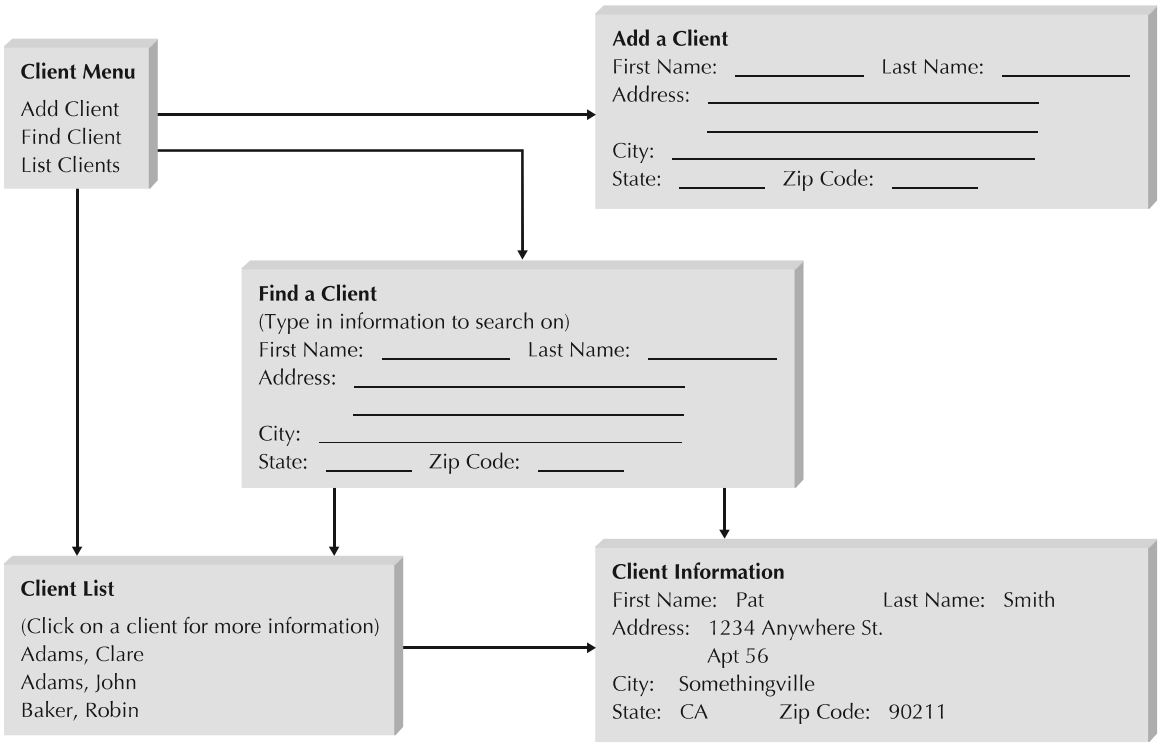



**FIGURE 10-6** Sample Storyboard

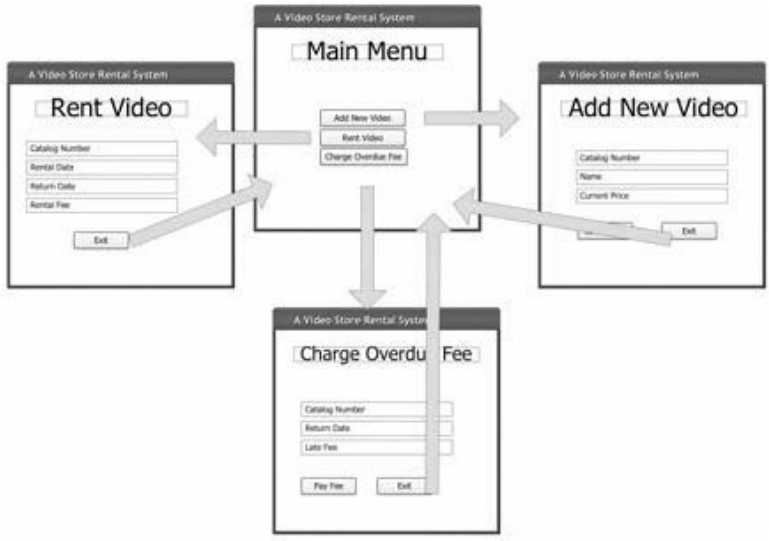



**FIGURE 10-7**
Sample Combined Windows Navigation and Layout Diagrams

the look and feel of the evolving system without having to actually implement anything, by combining the windows layout diagrams with the windows navigation diagram into a single better storyboard type of diagram (see Figure 10-7).

**User Interface Prototypes** With today's programming environments, such as Visual Studio and NetBeans, it is fairly easy to develop executable prototypes (see Figure 10-8) of the user interface that would allow the user to be able to interact with the user interface by clicking on buttons and entering pretend data into forms (but because there is no system behind the pages, the data are never processed). The different parts of the user interface are linked together so that as the user clicks on buttons, the requested part of the system appears. These executable prototypes take longer to develop than windows navigation diagrams, windows layout diagrams, and storyboards but have the distinct advantage of showing *exactly* what the screens will look like. The user does not have to guess about the shape or position of the elements on the screen. However, one of the potential issues that can arise when developing user interface prototypes is that the user's expectations of when the systems will be completed can become unrealistic. To actually connect the prototype up to the problem domain such that the system actually works is not a trivial problem. So, user expectations need to be carefully managed. Otherwise, a system that meets all of its specifications could end up being considered a failure.

**Selecting the Appropriate Techniques** Projects often use a combination of different interface design prototyping techniques for different parts of the system. Storyboarding is the fastest and least expensive but provides the least amount of detail. Windows layout diagrams provide more of a feel that the user will experience, while remaining fairly inexpensive to develop. User interface prototypes are the slowest, most expensive, and most detailed approach. Therefore, storyboarding is used for parts of the system in which the interface is well understood and when more-expensive prototypes are thought to be unnecessary. However, in most cases it is probably worth the additional cost of developing windows layout diagrams in addition to storyboards. User interface prototypes are used for parts of the system that are critical, yet not well understood.

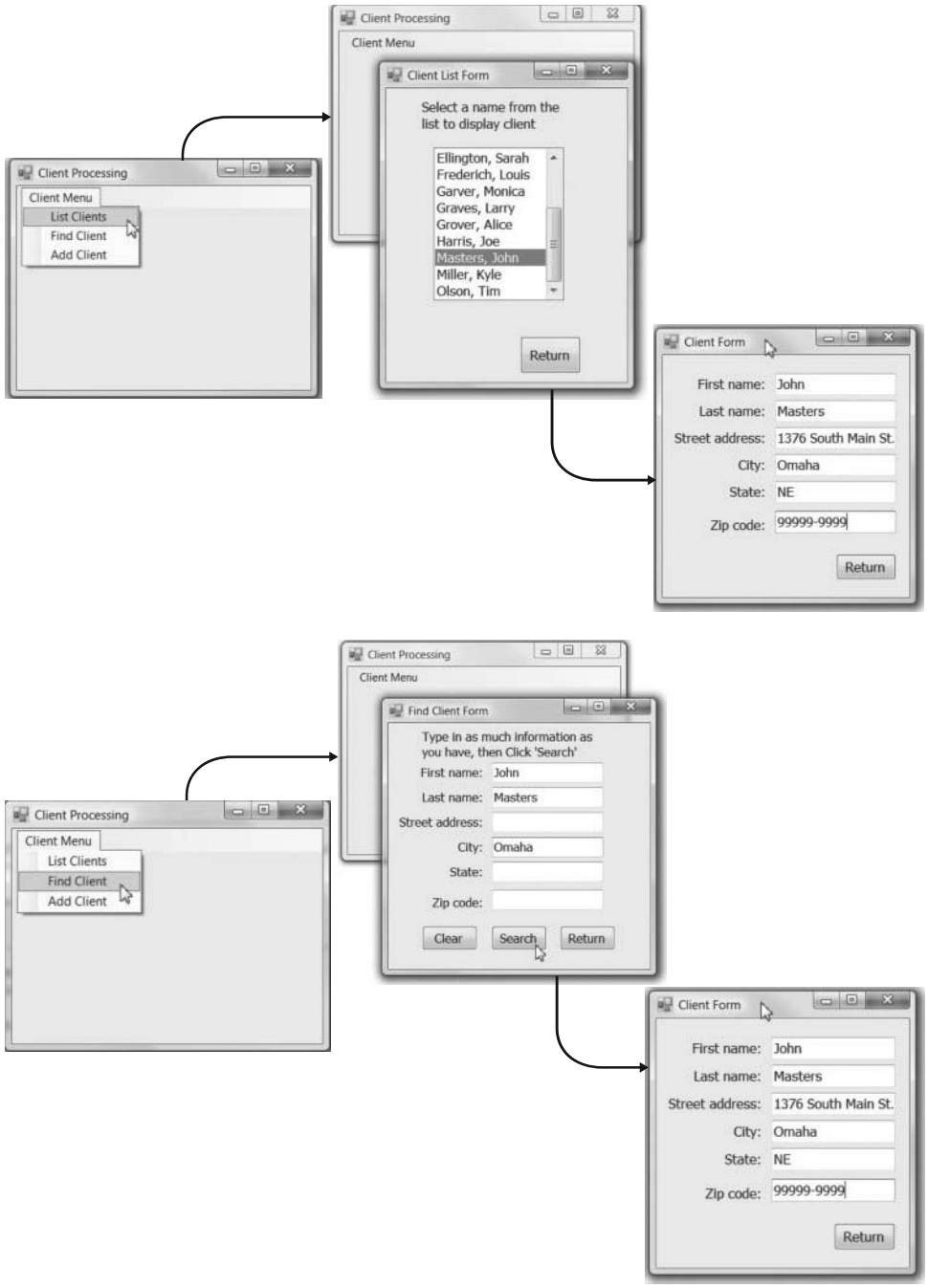



**FIGURE 10-8**
Sample User Interface Prototype

## Interface Evaluation[7]

The objective of interface evaluation is to understand how to improve the interface design before the system is complete. Most interface designers intentionally or unintentionally design an interface that meets their personal preferences, which might or might not match

[7] Verifying and validation approaches, in general, were described in Chapters 4 through 7. Also, further approaches to testing the evolving system are described in Chapter 12. In this section, we describe approaches that have been customized to the human–computer interaction layer.

| | |
|---|---|
| VP & EXECUTIVE PUBLISHER: | Don Fowley |
| EXECUTIVE EDITOR: | Beth Lang Golub |
| CONTENT EDITOR: | Mary O'Sullivan |
| ASSOCIATE EDITOR: | Ellen Keohane |
| MARKETING MANAGER: | Christopher Ruel |
| ASSOCIATE PRODUCTION MANAGER: | Joyce Poh |
| DESIGNER: | Wendy Lai |

Cover Image: © Christopher Boswell/Shutterstock

This book was set in 10/12 Minion pro by Aptara and printed and bound by Courier Kendallville. The cover was printed by Courier Kendallville.

This book is printed on acid-free paper. ♾

Founded in 1807, John Wiley & Sons, Inc. has been a valued source of knowledge and understanding for more than 200 years, helping people around the world meet their needs and fulfill their aspirations. Our company is built on a foundation of principles that include responsibility to the communities we serve and where we live and work. In 2008, we launched a Corporate Citizenship Initiative, a global effort to address the environmental, social, economic, and ethical challenges we face in our business. Among the issues we are addressing are carbon impact, paper specifications and procurement, ethical conduct within our business and among our vendors, and community and charitable support. For more information, please visit our website: www.wiley.com/go/citizenship.

Copyright © 2015, 2012, 2009 John Wiley & Sons, Inc. All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning or otherwise, except as permitted under Sections 107 or 108 of the 1976 United States Copyright Act, without either the prior written permission of the Publisher, or authorization through payment of the appropriate per-copy fee to the Copyright Clearance Center, Inc., 222 Rosewood Drive, Danvers, MA 01923 (Web site: www.copyright.com). Requests to the Publisher for permission should be addressed to the Permissions Department, John Wiley & Sons, Inc., 111 River Street, Hoboken, NJ 07030-5774, (201) 748-6011, fax (201) 748-6008, or online at: www.wiley.com/go/permissions.

Evaluation copies are provided to qualified academics and professionals for review purposes only, for use in their courses during the next academic year. These copies are licensed and may not be sold or transferred to a third party. Upon completion of the review period, please return the evaluation copy to Wiley. Return instructions and a free of charge return shipping label are available at: www.wiley.com/go/returnlabel. If you have chosen to adopt this textbook for use in your course, please accept this book as your complimentary desk copy. Outside of the United States, please contact your local sales representative.

***Library of Congress Cataloging-in-Publication Data***

Dennis, Alan.

Systems analysis & design: an object-oriented approach with UML/Alan Dennis, Indiana University, Barbara Haley Wixom, Massachusetts Institute of Technology, David Tegarden, Virginia Tech; with contributions by Elaine Seeman, East Carolina University.–Fifth edition.

pages cm

Includes bibliographical references and index.

ISBN 978-1-118-80467-4 (pbk. : alk. paper)

1. System analysis. 2. System design. 3. UML (Computer science) I. Wixom, Barbara Haley, 1969-II. Tegarden, David Paul. III. Seeman, Elaine. IV. Title. V. Title: System analysis and design.

QA402.D395 2015

004.2'1–dc23

2014048338

**Printed in the United States of America**

10 9 8 7 6 5 4 3 2 1